REQMEMBENCH: BENCHMARKING TEMPORAL REQUIREMENT-STATE RECONSTRUCTION FOR CODING AGENTS

Anonymous authors
Paper under double-blind review

ABSTRACT

This paragraph will be written last and must remain a single paragraph. It should answer six questions in order: how the use of coding agents is changing; what existing coding evaluations do not explicitly measure; what new evaluation capability this paper defines; how ReqMemBench operationalizes that capability; how large the benchmark is; and what the most important empirical findings are. The intended narrative is that coding agents are becoming persistent participants in software projects, real requirements evolve through repeated revision and implementation feedback, existing benchmarks do not explicitly evaluate reconstruction of the currently valid requirement state, and ReqMemBench fills this gap with temporal requirement-state gold annotations. *[Insert final benchmark scale and the main empirical findings.]*

1 INTRODUCTION

This section should establish that correctly continuing a real software project requires more than understanding the current task and code. An agent must also determine which historically expressed requirements remain valid at the present time. The argument should proceed from the temporal nature of real projects, through the need to reconstruct current requirements, to the absence of an explicit temporal requirement-state target in existing coding evaluations. The introduction should comprise five or six natural paragraphs rather than formal subsections.

1. **From isolated coding tasks to stateful software projects.** Explain that coding agents are evolving from one-shot code generators into persistent participants in software projects, and that an agent may enter a project at any intermediate time. A requirement may evolve as
$$\text{INTRODUCE} \rightarrow \text{MODIFY} \rightarrow \text{DEFER} \rightarrow \text{RESUME} \rightarrow \text{REMOVE},$$
or as
$$\text{AMBIGUOUS} \rightarrow \text{CLARIFY} \rightarrow \text{RESOLVE}.$$
Consequently, different time points correspond to different valid specifications, and the desired action is generally not a function of the current task alone:
$$\text{DesiredAction}_{t^*} \neq f(q_{t^*}), \qquad \text{DesiredAction}_{t^*} = f(H_{<t^*}, C_{t^{*-}}, q_{t^*}).$$
2. **The missing state between history and coding.** Distinguish between what happened and what remains valid now. If a requirement was introduced, modified, deferred, and then removed, the correct interpretation is not merely to retain four messages, but to infer
$$\text{State}_X(t_4) = \text{REMOVED}.$$
The central conceptual statement is: the challenge is not to remember everything that happened, but to determine what still matters now.
3. **The gap in existing coding evaluation.** Summarize conventional repository-level evaluation as
$$(C, q) \rightarrow \text{Patch},$$

Figure 1 placeholder: Motivation and task definition

Introduce A $\rightarrow$ Modify A $\rightarrow$ Introduce B $\rightarrow$ Remove B $\rightarrow$ Clarify A

$\downarrow\ t^*\ \downarrow$

Current code + current task $\rightarrow$ agent $\rightarrow$ current valid requirement state $\rightarrow$ correct action

Figure 1: The ReqMemBench takeover setting. An agent enters a project at an intermediate time and reconstructs the currently valid requirement state from the preceding history, pre-task code state, and current task.

where correctness is primarily defined by whether a model resolves a given issue in a given repository. In contrast, ReqMemBench studies

$$(H, C, q) \rightarrow \text{CurrentRequirementState} \rightarrow \text{Action}.$$

The claim should not be that existing benchmarks contain no history. The more precise claim is that they generally define correctness around the current task outcome rather than an explicit, temporally evolving requirement state. The problem is also distinct from simple long-context retrieval: even when all relevant messages are accessible, an agent must reconcile revisions, invalidations, deferrals, resumptions, clarifications, and execution feedback.

4. **The evaluation target.** Introduce the core formulation:

$$\boxed{H_{<t^*} + C_{t^{*-}} + q_{t^*} \rightarrow G_P(t^{*-}) \rightarrow \text{Action}}.$$

ReqMemBench does not assume that the same agent has participated since the first day of a project. It evaluates a more focused takeover setting: when a coding agent enters a real project at an arbitrary intermediate time, can it reconstruct what the project currently requires and continue development correctly?

5. **The ReqMemBench approach.** Present the construction pipeline at a high level without introducing serialization details:

$$\text{RealProjectHistory} \rightarrow \text{RequirementEvents} \rightarrow \text{RequirementStateGraph} \rightarrow \text{GoldState}(t^*) \rightarrow \text{EvaluationInstar}$$

State that the data come from real software projects, requirement evolution is grounded in source messages, event replay constructs temporal state, a single project can yield instances at multiple takeover points, each takeover point has an explicit pre-task gold state, and the target task determines the correct subsequent action.

6. **Contributions.** Fix the final contribution list to four items:

   - **A new evaluation problem.** Formalize temporal requirement-state reconstruction for coding agents: recovering the currently valid project requirements at an arbitrary intermediate time.
   - **A real-world benchmark.** Construct ReqMemBench from longitudinal software projects, authentic client–developer histories, evolving requirements, intermediate code states, and real target tasks.
   - **Temporal gold-state construction.** Introduce

     $$\text{Messages} \rightarrow \text{Events} \rightarrow \text{StateGraph} \rightarrow G_P(t),$$

     thereby providing explicit temporal requirement ground truth rather than only a collection of historical messages.
   - **Empirical findings.** Use controlled No-History, Full-History, and Oracle-History evaluations to identify bottlenecks in evidence access, context management, temporal state reasoning, and downstream execution. *[Replace this description with concrete findings after completing the experiments.]*

2 Related Work

This section should establish that coding benchmarks, long-term memory benchmarks, and long-horizon conversational coding benchmarks each cover part of the problem, but none uses the temporal requirement state defined by ReqMemBench as its explicit evaluation object. The comparison should focus on differences in inputs, gold targets, and outputs rather than understating the contributions of prior work. *[Add and verify all citations before submission.]*

2.1 Coding Benchmarks

Organize this subsection around the progression

$$\text{FunctionGeneration} \rightarrow \text{RepositoryIssueResolution} \rightarrow \text{AgenticSoftwareEngineering}.$$

Begin with function-generation benchmarks such as HumanEval and MBPP. Then discuss SWE-bench and related work that extends evaluation to issue resolution in real repositories. Continue with project-level and agentic software-engineering benchmarks, including process-oriented evaluations such as RigorBench. Conclude that these benchmarks have progressively expanded how an agent completes a task, while correctness is still generally centered on the outcome of a given task rather than an explicit project requirement state at an arbitrary time.

2.2 Long-Term Memory and Long-Context Evaluation

Review benchmarks such as LongMemEval, which evaluate information extraction, multi-session reasoning, temporal reasoning, knowledge updates, and abstention. Explain how these abilities relate to ReqMemBench while emphasizing the difference in evaluation object. A typical memory benchmark takes the form

$$\text{History} + \text{Query} \rightarrow \text{Answer},$$

whereas ReqMemBench evaluates

$$\text{ProjectHistory} + \text{Code} + \text{Task} \rightarrow \text{ProjectState} \rightarrow \text{DevelopmentAction}.$$

Here, history is not merely a knowledge base to be queried; it is temporal evidence from which the current project specification must be constructed.

2.3 Long-Horizon Conversational Coding

Use LoCoEval as the main comparison in this subsection. Describe its focus on repository-oriented long-horizon conversational context management, iterative requirements, and information retention. The intended distinction is not that one problem subsumes the other: LoCoEval asks how long conversational context should be managed in repository-oriented interaction, whereas ReqMemBench asks which requirements remain valid in an evolving real project and what the agent should do next. Verify the data-generation setting, task types, annotation targets, and whether requirement lifecycle, scope, ambiguity, execution state, and arbitrary takeover points are explicitly represented.

2.4 Positioning ReqMemBench

Table 1 should summarize the complementary scope of the four research directions. Its purpose is not to mark every competing category as lacking every feature, but to make the distinct evaluation object of ReqMemBench immediately visible.

3 Task Formulation: Requirement-State-Aware Project Continuation

This section should establish that the currently valid requirement state is a well-defined and measurable evaluation object with a meaningful relationship to downstream development actions. It defines the temporal project boundary, requirement atoms and their state, the project-level gold state, state recovery through event replay, and the two evaluation levels of state reconstruction and state-conditioned action.

Table 1: Positioning ReqMemBench relative to related evaluation directions. All entries must be verified against the cited work.

Dimension	Coding	Memory	Conv. coding	ReqMemBench
Real repository	*[TBD]*	*[TBD]*	*[TBD]*	Yes
Longitudinal project	*[TBD]*	*[TBD]*	*[TBD]*	Yes
Historical conversation	*[TBD]*	*[TBD]*	*[TBD]*	Yes
Requirement evolution	*[TBD]*	*[TBD]*	*[TBD]*	Yes
Explicit lifecycle	*[TBD]*	*[TBD]*	*[TBD]*	Yes
Scope and ambiguity	*[TBD]*	*[TBD]*	*[TBD]*	Yes
Temporal gold state $G(t)$	*[TBD]*	*[TBD]*	*[TBD]*	Yes
Intermediate takeover	*[TBD]*	*[TBD]*	*[TBD]*	Yes
Downstream action	*[TBD]*	*[TBD]*	*[TBD]*	Yes

3.1 Temporal Project Setting

Define a project as

$$P = (H, C),$$

where H is the timestamped project history and C is the code state as it evolves over time. For a target task q_{t^*} arriving at time t^*, the agent may use only the preceding history

$$H_{<t^*}$$

and the code snapshot immediately before the task arrives,

$$C_{t^*-}.$$

Specify which client messages, developer messages, and execution feedback belong to H; how code snapshots are aligned with time; how t^* is determined; and why all evidence after t^* is excluded to prevent future-information leakage.

3.2 Requirement and Requirement State

Use a requirement atom as the independent lifecycle unit. A requirement family is only a semantic grouping and does not have an independent lifecycle. For a requirement atom r, define

$$G_r(t) = \big(\text{Attributes}, \text{Lifecycle}, \text{Scope}, \text{Ambiguity}, \text{Execution}\big).$$

The five dimensions represent the current requirement value; a lifecycle such as ACTIVE, DEFERRED, REMOVED, or CLARIFY; persistence, component, and contextual scope; open or closed ambiguity and the affected dimensions; and execution states such as FAILED, CLAIMED_WORKING, and VERIFIED_WORKING. *[Provide complete state vocabularies and the requirement-atom segmentation rules.]*

3.3 Project-Level Gold State

Define the project-level requirement state at time t as

$$G_P(t) = \{G_r(t) \mid r \text{ appeared before } t\}.$$

A removed requirement does not disappear from the gold state. It remains represented with lifecycle REMOVED because recognizing that a requirement is invalid, and avoiding its continued implementation or accidental resurrection, is itself part of the evaluated capability.

3.4 Requirement State as Event Replay

Convert the history relevant to requirement r into an ordered event sequence

$$E_r = (e_1, e_2, \ldots, e_n),$$

Figure 2 placeholder: ReqMemBench construction pipeline

Raw project → event annotation → state graph → target selection
→ temporal gold state → evaluation instance

Figure 2: The ReqMemBench construction pipeline from real project histories to temporal requirement-state evaluation instances.

and recover the complete state at any time through

$$G_r(t) = \mathrm{Replay}(E_r^{\leq t}).$$

Events record what changed and why, while state nodes represent the complete requirement state after each change. This design reflects four properties: requirements are evolving objects; messages are evidence rather than states; events retain temporal provenance; and the same project graph can be replayed at multiple takeover points.

3.5 Evaluation Task

The complete ReqMemBench task is

$$H_{<t^*} + C_{t^{*-}} + q_{t^*} \rightarrow \widehat{G}_P(t^{*-}) \rightarrow \widehat{A}_{t^*}.$$

Evaluation has two levels. Level I evaluates state reconstruction:

$$\widehat{G}_P(t^{*-}) \stackrel{?}{=} G_P(t^{*-}).$$

Level II evaluates the state-conditioned action:

$$\widehat{A}_{t^*} \stackrel{?}{=} A^*_{t^*}.$$

This separation distinguishes an agent that does not know what the project currently requires from an agent that understands the requirements but fails to execute them correctly.

4 ReqMemBench Construction

This section should establish that the ReqMemBench gold annotations are systematically recovered from naturally occurring project histories rather than created as artificial tasks for evaluation. It should be one of the most detailed sections of the paper, covering data sources, event annotation, state-graph construction, target-time selection, instance generation, quality control, and benchmark statistics.

4.1 Data Source and Project Selection

Explain why real freelance software projects are suitable for studying naturally occurring requirement evolution. List the available artifacts, such as client–developer messages, code snapshots, commits or deliveries, execution feedback, and timestamps. Define inclusion and exclusion criteria based on historical completeness, code recoverability, meaningful interaction length, identifiable requirement evolution, and privacy or authorization constraints. Emphasize that the introduction, modification, deferral, removal, and clarification of requirements occurred during real development rather than being inserted by benchmark authors. *[Insert the data source, authorization procedure, filtering pipeline, and final project count.]*

4.2 Stage 1: Requirement Event Annotation

The first construction stage is

$$\mathrm{RawMessages} \rightarrow \mathrm{RequirementFamilies} \rightarrow \mathrm{RequirementAtoms} \rightarrow \mathrm{RequirementEvents}.$$

The event taxonomy should include INTRODUCE, MODIFY, REMOVE, DEFER, RESUME, AMBIGUOUS, CLARIFY, RESOLVE, IMPLEMENTATION_CLAIM, RUNTIME_FAILURE, and RUNTIME_VERIFICATION. Explain why these events capture changes in value, lifecycle, scope, ambiguity, and execution status, and require every event to retain source-message evidence. Detailed serialization fields can be placed in the appendix.

4.3 Stage 2: Requirement State Graph

The second stage converts events into state nodes and a state graph:

$$\text{Events} \rightarrow \text{StateNodes} \rightarrow \text{StateGraph}.$$

Each node stores a complete requirement state, and each edge corresponds to the event responsible for the transition. For example,

$$S_1 \xrightarrow{\text{MODIFY}} S_2 \xrightarrow{\text{DEFER}} S_3.$$

Explain why an explicit graph is preferable to an event list alone, how replay deterministically reconstructs state, and how ambiguity and execution state are updated alongside lifecycle. *[Add the complete state-transition rules or transition table.]*

4.4 Target Time and Task Selection

Define the target time as

$$t^* = \text{arrival time of the target client task.}$$

Each target task establishes two boundaries: the pre-task state $G_P(t^{*-})$ and the state after the task has been correctly interpreted and handled, $G_P(t^{*+})$. The current task triggers the transition

$$G_P(t^{*-}) \xrightarrow{q_{t^*}} G_P(t^{*+}).$$

The pre-task state is what the agent should recover upon takeover. The post-task state, affected requirements, and task events jointly help define the correct action. *[Specify target-task eligibility and temporal-alignment rules.]*

4.5 Instance Construction

The basic unit of the benchmark is a target task at a target time, not an individual requirement, because one client message may affect multiple requirements. Each instance contains the historical record $H_{<t^*}$, pre-task code state $C_{t^{*-}}$, and current task q_{t^*} as inputs. Hidden gold annotations contain the relevant historical evidence, $G_P(t^{*-})$, affected requirements, task events, $G_P(t^{*+})$, and expected action. *[Include one complete serialized instance in the appendix.]*

4.6 Annotation and Quality Control

Clearly separate the responsibilities of language models, deterministic code, and human reviewers:

- **Language-model assistance:** propose requirement families, requirement atoms, event types, and source evidence; treat all generated outputs as candidates for review.
- **Deterministic processing:** sort events by time, replay events, generate states, validate cross-instance consistency, detect future leakage, and reject impossible transitions.
- **Human review:** verify requirement boundaries, event semantics, scope, ambiguity, execution status, and expected actions; adjudicate disagreements.
- **Evidence retention:** trace every event and state transition to a source message and timestamp.
- **Quality reporting:** report the manual-review sample, double-annotation or adjudication protocol, agreement by field, and appropriate agreement statistics.

[Insert the actual annotation workflow, review scale, agreement results, and error-correction procedure.]

4.7 Benchmark Statistics

Use Table 2 and at least one distribution figure to describe the benchmark at project, requirement, event, and target-task levels. Report totals together with means, medians, ranges, or quantiles so that scale is not represented by a single aggregate count.

Table 2: Planned ReqMemBench statistics.

Level	Statistics	Value
Project	Count, duration, messages, history length, repository size	*[TBD]*
Requirement	Families, atoms, and requirements per project	*[TBD]*
Event	Count, type distribution, events per requirement, transitions	*[TBD]*
Task	Count, affected requirements, position, preceding history length	*[TBD]*

5 Experimental Design and Evaluation Protocol

This section should establish that the experimental design separates general coding ability, access to historical evidence, requirement-state reasoning, and downstream action. All controlled conditions should use the same agent scaffold, tool permissions, and resource budgets, varying only the explicitly defined independent variable—most importantly, access to project history.

Table 3 crosses five experimental dimensions: history difficulty, research question, coding-agent framework, backbone model, and controlled input condition. The two stacked panels group results first by agent framework, then by backbone model, and finally by input condition. Rows stratify instances by the number of conversation turns before the target task and by the requirement-management capability being evaluated. Empty cells reserve locations for future measurements. Dashes mark combinations that are not applicable: RQ1 requires access to the full project history and is therefore evaluated only under C2.

5.1 Research Questions

Organize the experiments around four top-level research questions that follow the requirement-management pipeline from evidence selection to code execution:

- **RQ1: Selection.** Can an agent identify the historical requirements and evidence relevant to the target task from the full project history?
- **RQ2: State Reconstruction.** Can an agent reconcile requirement evolution and recover the currently valid requirement state at the takeover point?
- **RQ3: Memory-or-Clarify.** Can an agent determine whether the available project memory resolves the task or whether unresolved ambiguity requires clarification?
- **RQ4: Requirement-to-Code.** Can an agent translate the correct requirement state into an appropriate development action and, where execution is supported, a correct code change?

5.2 History Conditions

Use three controlled input conditions:

- **C1—No History:**
$$I_{\mathrm{C1}}(t^*) = C_{t^*-} + q_{t^*}.$$
This condition measures how much performance is supported by the current code and task alone.
- **C2—Full History:**
$$I_{\mathrm{C2}}(t^*) = H_{<t^*} + C_{t^*-} + q_{t^*}.$$
This condition represents the realistic takeover setting and includes relevant, stale, and irrelevant information.
- **C3—Oracle Relevant History:**
$$I_{\mathrm{C3}}(t^*) = H^{\mathrm{rel}}_{<t^*} + C_{t^*-} + q_{t^*}.$$
This condition supplies only human-verified relevant history, largely removing selection and retrieval burden while retaining temporal reconciliation and state reasoning.

Table 3: ReqMemBench experimental matrix across agent frameworks, backbone models, history-length strata, research questions, and input conditions. C1: No History; C2: Full History; C3: Oracle Relevant History. RQ1 is evaluated only under C2. The two panels preserve the full experimental matrix while retaining readable body text.

Difficulty	Research question	Codex								
		GPT-5.6 SOL			GPT-5.6 Terra			GPT-5.5		
		C1	C2	C3	C1	C2	C3	C1	C2	C3
Short (0–25)	RQ1 Select.	–		–	–		–	–		–
	RQ2 Recon.									
	RQ3 Clarify									
	RQ4 Execute									
Medium (26–50)	RQ1 Select.	–		–	–		–	–		–
	RQ2 Recon.									
	RQ3 Clarify									
	RQ4 Execute									
Long (> 50)	RQ1 Select.	–		–	–		–	–		–
	RQ2 Recon.									
	RQ3 Clarify									
	RQ4 Execute									

Difficulty	Research question	Claude Code								
		Claude Opus 5			Claude Sonnet 5			Claude Haiku 4.5		
		C1	C2	C3	C1	C2	C3	C1	C2	C3
Short (0–25)	RQ1 Select.	–		–	–		–	–		–
	RQ2 Recon.									
	RQ3 Clarify									
	RQ4 Execute									
Medium (26–50)	RQ1 Select.	–		–	–		–	–		–
	RQ2 Recon.									
	RQ3 Clarify									
	RQ4 Execute									
Long (> 50)	RQ1 Select.	–		–	–		–	–		–
	RQ2 Recon.									
	RQ3 Clarify									
	RQ4 Execute									

RQ1 is defined only for C2: C1 contains no historical context to select, while C3 has already resolved selection by providing oracle-relevant history. For RQ2–RQ4, the C2–C1 difference estimates the benefit of project history, while the C3–C2 difference diagnoses losses associated with full-history selection, retrieval, and context management. Errors that remain under C3 indicate difficulties with state reconciliation, clarification decisions, or execution. The agent scaffold must remain fixed across conditions.

5.3 Benchmark Curation Validity

5.3.1 Requirement determinacy

≥ 200 steps of state changes and evaluate by human and other two LLMs at the same time. Calculate Cohen's/ Feiss's k)

5.3.2

sectionHistory Necessity Rate Whether the change is really necessary for coding agents. (complete chat history + change) vs. (abstract of final requirement only) comparing the different pass rate.

5.4 Models and Coding Agents

Report every evaluated model and version, the agent harness, context window, available tools, execution budget, temperature, retry policy, stopping criteria, and evaluation date. Treat the model–harness combination as the deployed evaluation unit while using a common scaffold to isolate the effect of history condition. *[Insert final models, versions, prompts, tools, budgets, and number of repeated runs.]*

5.5 Metrics

For RQ1, report relevant-requirement and evidence-selection precision, recall, and F1 under C2. Define the RQ2 state reconstruction score over six dimensions:

$$S_{\text{state}} = f(S_{\text{selection}}, S_{\text{value}}, S_{\text{lifecycle}}, S_{\text{scope}}, S_{\text{ambiguity}}, S_{\text{execution}}).$$

Compute state-dimension scores at the requirement level, aggregate them at the task level, and use project-level macro averaging so that projects with more requirements or more annotated fields do not receive disproportionate weight. For RQ3, report the accuracy and class-conditional performance of the memory-versus-clarify decision, including unsupported autonomous decisions and unnecessary clarification requests. For RQ4, evaluate action correctness across implement, modify, remove, preserve, and clarify; for executable repositories, additionally report test success, regressions, and task-specific correctness. Report all metrics by history-length stratum and input condition where applicable. *[Specify matching rules, partial credit, macro-averaging equations, confidence intervals, and significance tests.]*

6 Results

Organize this section directly around the four research questions rather than using generic "main results" and "additional results" headings. Each subsection should begin with a one-sentence answer to its RQ, present the primary table or figure, and then explain the most important differences, failure dimensions, and statistical uncertainty. The current outline reserves result locations without presupposing the findings.

6.1 RQ1: Can Agents Select the Relevant Historical Requirements?

Evaluate requirement and evidence selection under C2 Full History for each agent–model configuration and history-length stratum. Report selection precision, recall, and the task- or project-level aggregate defined in Section 5. Analyze whether selection quality degrades from short to medium and long project histories. *[Insert the RQ1 selection results and the first concrete finding.]*

6.2 RQ2: Can Agents Reconstruct the Current Requirement State?

Report overall state-reconstruction performance and performance on all six state dimensions under C1, C2, and C3. Answer how well current agents perform, which state dimension is most difficult, where the largest differences between models or agents arise, and how history difficulty changes these conclusions. Table 4 is the primary state-reconstruction result table. *[Insert the main RQ2 finding at the start of this subsection.]*

Table 4: RQ2 requirement-state reconstruction results. Report project-level macro averages with uncertainty estimates.

Model	Overall	Select.	Value	Life.	Scope	Ambig.	Exec.
[Model]	–	–	–	–	–	–	–
[Model]	–	–	–	–	–	–	–

Compare C1 No History, C2 Full History, and C3 Oracle Relevant History within the RQ2 analysis. $\mathrm{C3} > \mathrm{C2} > \mathrm{C1}$ indicates that history is useful but selection or retrieval remains a bottleneck. $\mathrm{C3} > \mathrm{C1} > \mathrm{C2}$ indicates that unfiltered history may harm the agent. $\mathrm{C3} \approx \mathrm{C2} \ll 100\%$ indicates that temporal state reasoning, rather than retrieval, is the dominant limitation. *[Insert the central C1–C2–C3 comparison and select the interpretation supported by the data.]*

6.3 RQ3: Can Agents Decide When to Use Memory or Clarify?

Evaluate whether the agent correctly proceeds from available project memory or requests clarification when ambiguity remains open. Break down accuracy by ambiguity type, history-length stratum, and input condition. Distinguish unsupported autonomous decisions from unnecessary clarification requests, and test whether C3 reduces selection errors without eliminating genuine ambiguity. *[Insert the RQ3 decision results and representative cases.]*

6.4 RQ4: Can Agents Translate Requirements into Correct Code?

Analyze the relationship between state accuracy and downstream action or code accuracy, with particular attention to

$$P(\mathrm{ActionCorrect} \mid \mathrm{StateCorrect}) \quad \text{versus} \quad P(\mathrm{ActionCorrect} \mid \mathrm{StateWrong}).$$

Identify which state errors most often propagate into action or coding errors, and analyze both "incorrect state but accidentally correct action" and "correct state but incorrect execution" cases. This analysis tests whether $G_P(t)$ is a meaningful bridge between project memory and coding outcomes rather than an artificial intermediate variable. *[Insert the RQ4 state-accuracy versus action-success figure.]*

7 Analysis and Discussion

This section should establish that ReqMemBench reveals concrete failure mechanisms of persistent coding agents, not merely a model ranking. The analysis should explain phenomena observed in the main experiments and should avoid presenting unsupported conclusions as established findings.

7.1 Requirement-Evolution Failure Modes

Break down errors by transitions involving INTRODUCE, MODIFY, REMOVE, DEFER, RESUME, AMBIGUOUS, CLARIFY, and RESOLVE. Initial error categories include stale-state errors that retain superseded values, resurrection errors that reuse removed requirements, missing-scope errors that apply a valid requirement to the wrong component or context, ambiguity hallucinations that resolve underspecified choices without evidence, and execution-state confusion that treats an implementation claim as verified success. *[Revise the taxonomy using observed errors and add anonymized examples.]*

7.2 Effects of History Characteristics

Analyze history length, evidence distance, relevant-evidence density, number of requirement updates, number of parallel requirements, target position within the project, and number of affected requirements per task. The goal is to determine which properties make long-running projects difficult to take over. *[Specify bins, controls, and the statistical analysis.]*

7.3 Is More Context Better?

Compare Full History directly with No History. If Full History is not consistently superior, examine noise, stale state, conflicting evidence, and context competition as potential mechanisms, and emphasize that context availability and state understanding are different capabilities. If Full History consistently helps, report which state dimensions and task types account for the improvement.

7.4 Coding Ability versus Requirement-State Ability

Compare model rankings on a standard coding benchmark, ReqMemBench State, and ReqMemBench Action. If a model that performs better on standard coding performs worse on ReqMemBench, use the ranking inversion to motivate the claim that isolated-task coding performance may not reliably predict effectiveness in persistent project settings. If no inversion occurs, report the observed relationship without overstating the distinction. *[Specify the source of external coding scores, version alignment, and correlation analysis.]*

7.5 Implications for Coding-Agent Design

Connect the results to five system-design implications: memory should represent current state rather than only retrieve messages; forgetting and invalidation are as important as remembering; requirement memory should preserve temporal provenance; open ambiguity should trigger clarification; and repository state and conversational memory should be interpreted jointly. Each implication should be tied to a reported result or failure case.

8 Limitations

This section should explicitly delimit the conclusions supported by ReqMemBench. It should cover at least the following six limitations:

- **Data domain.** Freelance projects do not represent every enterprise development process, organizational structure, or quality requirement.
- **Historical takeover setting.** The benchmark provides an already observed history and evaluates project-state reconstruction from available evidence. It does not have the same agent experience every interaction online from t_0, and therefore does not directly evaluate an online memory-writing policy.
- **Requirement-annotation subjectivity.** Requirement atomicity, scope, and ambiguity permit reasonable interpretation differences that must be controlled through human validation and agreement reporting.
- **Code availability and executability.** Dependencies, environments, or external services may not be fully recoverable for every real project, so not every instance can support end-to-end execution tests.
- **Requirement gold versus implementation gold.** A requirement state may permit multiple valid implementations; therefore,

$$\mathrm{RequirementGold} \neq \mathrm{UniqueCodePatch}.$$

- **Coverage.** The current data may not cover enterprise multi-team development, multi-source histories spanning issue trackers, chat, and code review, multi-year project memory, or autonomous agents operating continually online.

9 Conclusion

The conclusion should be brief and answer three questions. First, what was missing: existing coding evaluation provides an incomplete picture of agents that must operate inside long-running software projects. Second, what was introduced: ReqMemBench evaluates whether an agent can reconstruct

the currently valid requirement state at arbitrary project times and continue development accordingly. Third, what was learned: summarize the one or two most important empirical findings. *[Insert the final findings.]* End with the central message: for persistent coding agents, knowing how to code is not sufficient; they must also know what the project requires now.

AI USE STATEMENT

[Complete this required statement according to the ICLR 2027 AI Policy for Authors. Disclose the use of generative AI in research ideation, data processing, code, experiments, writing, or editing; explain how the authors reviewed AI-assisted work; and state that the authors take responsibility for the final paper.]

ETHICS STATEMENT

[Describe data authorization, privacy protection, de-identification, sensitive-information handling, release scope, potential harms, and legal or ethical safeguards for the real freelance project data.]

REPRODUCIBILITY STATEMENT

[Point to the sections, appendices, and supplementary artifacts that document data construction, annotation, event replay, evaluation protocols, model configurations, and statistical analysis.]

REFERENCES

A APPENDIX STRUCTURE

The appendix should contain the complete requirement-state schema and field definitions; event types and state-transition rules; project-selection and privacy procedures; annotation guidelines and review interfaces; complete evaluation prompts and output formats; model and agent configurations; metric equations and statistical tests; additional benchmark distributions; full results by project, event, and history characteristic; failure cases; and one end-to-end example from source messages to the state graph and target action.